

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 12

Тема: «Отладка программы с использованием специализированных средств
отладки»

Цель работы:

изучить способы отладки программы с использованием языка
программирования Python

Выполнила:
Позднякова Марина,
Студент 24-ИС

2026г.

Этап 1: Запуск и первичный анализ

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
```

```
===== RESTART: Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/1.py =====
```

```
=====
ЗАПУСК ПРОГРАММЫ (С ОШИБКАМИ)
=====
```

```
1. Однопоточный вызов:
```

```
✗ ОШИБКА: 'price'
```

```
Traceback (most recent call last):
```

```
File "Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/1.py", line 63, in <module>
```

```
    result = process_booking(test_data)
```

```
File "Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/1.py", line 28, in process_booking
```

```
    price = item['price'] # KeyError при отсутствии ключа!
```

```
~~~~~^~~~~~
```

```
KeyError: 'price'
```

```
=====
СТАТИСТИКА ПАМЯТИ:
=====
```

```
Топ-5 строк по потреблению памяти:
```

```
C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:388: size=15.7 KiB, count=177, average=91 B
```

```
C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\linecache.py:172: size=8603 B, count=86, average=100 B
```

```
<frozen importlib._bootstrap>:488: size=5740 B, count=56, average=102 B
```

```
<frozen abc>:123: size=2873 B, count=32, average=90 B
```

```
C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\threading.py:297: size=760 B, count=2, average=380 B
```

```
|
```

Тип ошибки: KeyError

Где: Строка 28, файл 1.py

Что произошло: Программа попыталась обратиться к ключу 'price' в словаре, но этого ключа не существовало.

Код:

```
import threading
```

```
import time
```

```
import tracemalloc
```

```
import math
```

```
# Глобальный кеш --- утечка памяти
```

```
CACHE = {}
```

```
def calculate_discount(price, nights):
```

```
    """
```

Расчет скидки с логической ошибкой:

- При `nights > 3`: скидка должна быть 20%, но формула неверная

```
"""
```

Логическая ошибка: скидка 20% ночью, но формула неправильная

```
if nights > 3:
```

```
    # Должно быть price * 0.2, а здесь price * (0.2 + nights)
```

```
    discount = price * (0.2 + nights) # Ошибка!
```

```
else:
```

```
    discount = 0
```

```
return discount
```

```
def process_booking(data):
```

```
    """
```

Обработка бронирований с многопоточным доступом

```
    """
```

Многопоточный доступ к общему CACHE

```
for item in data:
```

```
    price = item['price'] # KeyError при отсутствии ключа!
```

```
    nights = item['nights']
```

```
    # Утечка: кешируем все, даже если не нужно
```

```
    CACHE[f'{price}_{nights}'] = calculate_discount(price, nights)
```

```
return CACHE
```

```
def worker(data, thread_id):
```

```
    """Функция для потока"""
```

```
    print(f'Поток {thread_id} начал работу')
```

```

result = process_booking(data)

print(f'Поток {thread_id} завершил работу")

return result

if __name__ == "__main__":
    tracemalloc.start()

    # Тестовые данные с ошибкой
    test_data = [
        {'price': 1200, 'nights': 3},
        {'price': 500, 'nights': 5}, # Должна быть скидка 20%
        {'price': 100, 'nights': 2},
        {'vip': True, 'nights': 1}, # Ошибка: нет ключа 'price'
        {'price': 800, 'nights': 4}, # Должна быть скидка 20%
        {'price': 300, 'nights': 0}, # Деление на ноль? (не здесь)
    ]

    print("=" * 60)
    print("ЗАПУСК ПРОГРАММЫ (С ОШИБКАМИ)")
    print("=" * 60)

    try:
        # Однопоточный вызов
        print("\n1. Однопоточный вызов:")
        result = process_booking(test_data)
        print(f'Результат: {result}')

```

```
# Многопоточный вызов

print("\n2. Многопоточный вызов:")

threads = []

for i in range(3):
    t = threading.Thread(target=worker, args=(test_data, i))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

except Exception as e:
    print(f"\n ✖ ОШИБКА: {e}")
    import traceback
    traceback.print_exc()

# Вывод статистики памяти

print("\n" + "=" * 60)

print("СТАТИСТИКА ПАМЯТИ:")

print("=" * 60)

snapshot = tracemalloc.take_snapshot()

top_stats = snapshot.statistics('lineno')

print("\nТоп-5 строк по потреблению памяти:")

for stat in top_stats[:5]:
    print(stat)
```

Этап 2: Локализация с помощью breakpoint()

```
=====
🔧 ЗАПУСК ПРОГРАММЫ (С ОШИБКАМИ + breakpoint())
=====

📌 Инструкция по работе с pdb:
n - выполнить следующую строку
s - войти в функцию
c - продолжить до следующего breakpoint
p var - напечатать переменную
pp var - красивая печать
l - показать код вокруг текущей строки
q - выход
=====

📋 Тестовые данные:
0: {'price': 1200, 'nights': 3}
1: {'price': 500, 'nights': 5}
2: {'price': 100, 'nights': 2}
3: {'vip': True, 'nights': 1}
4: {'price': 800, 'nights': 4}
5: {'price': 300, 'nights': 0}

=====

1 ☐ ОДНОПОТОЧНЫЙ ВЫЗОВ:
=====

[ ] Остановка перед process_booking()
> y:\24 исип 2025\уп02\группа 2\позднякова м\день 12\2.py(126)<module>()
-> breakpoint() # Остановка перед вызовом функции
(Pdb)
```

Переменная item имеет неверное значение на 4-м элементе списка

Код:

```
"""
```

Вариант 1 — Отладка программы (Список словарей)

Полный код с breakpoint() для пошаговой отладки

```
"""
```

```
import threading
```

```
import time
```

```
import tracemalloc
```

```
import math
```

```
# Глобальный кеш --- утечка памяти
```

```
CACHE = {}
```

```
def calculate_discount(price, nights):
```

```
    """
```

```
    Расчет скидки с логической ошибкой:
```

```
    - При nights > 3: скидка должна быть 20%, но формула неверная
```

```
    """
```

```
    # Логическая ошибка: скидка 20% ночью, но формула неправильная
```

```
    if nights > 3:
```

```
        # Должно быть price * 0.2, а здесь price * (0.2 + nights)
```

```
        discount = price * (0.2 + nights) # Ошибка!
```

```
    else:
```

```
        discount = 0
```

```
    return discount
```

```
def process_booking(data):
```

```
    """
```

```
    Обработка бронирований с многопоточным доступом
```

```
    """
```

```
    # Многопоточный доступ к общему CACHE
```

```
for i, item in enumerate(data):
```

```
# =====
```

```
# ТОЧКА ОСТАНОВА 1: Проверка элемента перед обработкой
```

```
# =====
```

```
print(f"\n🕒 Обработка элемента {i}: {item}")
```

```
breakpoint() # Остановка здесь для исследования item
```

```
price = item['price'] # KeyError при отсутствии ключа!
```

```
nights = item['nights']
```

```
# =====
```

```
# ТОЧКА ОСТАНОВА 2: Проверка перед кешированием
```

```
# =====
```

```
print(f" price={price}, nights={nights}")
```

```
breakpoint() # Остановка перед записью в кеш
```

```
# Утечка: кешируем все, даже если не нужно
```

```
CACHE[f'{price}_{nights}'] = calculate_discount(price, nights)
```

```
# =====
```

```
# ТОЧКА ОСТАНОВА 3: Проверка после кеширования
```

```
# =====
```

```
print(f" Размер кеша: {len(CACHE)}")
```

```
breakpoint() # Остановка для проверки состояния кеша
```

```
return CACHE
```

```
def worker(data, thread_id):
```

```
    """Функция для потока"""
```

```
    print(f'🌀 Поток {thread_id} начал работу")
```

```
    # =====
```

```
    # ТОЧКА ОСТАНОВА 4: Вход в поток
```

```
    # =====
```

```
    breakpoint() # Остановка при входе в поток
```

```
    result = process_booking(data)
```

```
    print(f'✅ Поток {thread_id} завершил работу")
```

```
    return result
```

```
def monitor_memory():
```

```
    """Мониторинг использования памяти"""
```

```
    snapshot = tracemalloc.take_snapshot()
```

```
    top_stats = snapshot.statistics('lineno')
```

```

print("\n" + "=" * 60)

print("📊 СТАТИСТИКА ПАМЯТИ:")

print("=" * 60)

print("\nТоп-5 строк по потреблению памяти:")

for stat in top_stats[:5]:

    print(stat)


return top_stats


if __name__ == "__main__":

    # Запуск отслеживания памяти

    tracemalloc.start()


    print("=" * 60)

    print("🔪 ЗАПУСК ПРОГРАММЫ (С ОШИБКАМИ + breakpoint())")

    print("=" * 60)

    print("\n🔗 Инструкция по работе с pdb:")

    print("  n - выполнить следующую строку")

    print("  s - войти в функцию")

    print("  c - продолжить до следующего breakpoint")

    print("  p var - напечатать переменную")

    print("  pp var - красивая печать")

```

```
print(" l - показать код вокруг текущей строки")
```

```
print(" q - выход")
```

```
print("=" * 60)
```

```
# Тестовые данные с ошибкой
```

```
test_data = [
```

```
    {'price': 1200, 'nights': 3},
```

```
    {'price': 500, 'nights': 5}, # Должна быть скидка 20%
```

```
    {'price': 100, 'nights': 2},
```

```
    {'vip': True, 'nights': 1}, # Ошибка: нет ключа 'price'
```

```
    {'price': 800, 'nights': 4}, # Должна быть скидка 20%
```

```
    {'price': 300, 'nights': 0}, # Деление на ноль? (не здесь)
```

```
]
```

```
print("\n📋 Тестовые данные:")
```

```
for i, item in enumerate(test_data):
```

```
    print(f" {i}: {item}")
```

```
print("\n" + "=" * 60)
```

```
print("1️⃣ ОДНОПОТОЧНЫЙ ВЫЗОВ:")
```

```
print("=" * 60)
```

```
try:
```

```
# =====
```

```
# ТОЧКА ОСТАНОВА 5: Перед вызовом process_booking
```

```
# =====
```

```
print("\n⏸ Остановка перед process_booking()")
```

```
breakpoint() # Остановка перед вызовом функции
```

```
result = process_booking(test_data)
```

```
print(f"\n✅ Результат: {result}")
```

```
print(f"\n📦 Размер кеша: {len(CACHE)}")
```

```
except Exception as e:
```

```
print(f"\n❌ ОШИБКА: {e}")
```

```
import traceback
```

```
traceback.print_exc()
```

```
# =====
```

```
# ТОЧКА ОСТАНОВА 6: Перед многопоточным вызовом
```

```
# =====
```

```
print("\n" + "=" * 60)
```

```
print("2️⃣ МНОГОПОТОЧНЫЙ ВЫЗОВ:")
```

```
print("=" * 60)
```

```
print("\n⏸ Остановка перед созданием потоков")
```

```
breakpoint() # Остановка перед многопоточным вызовом
```

```
threads = []
```

```
for i in range(3):
```

```
    t = threading.Thread(target=worker, args=(test_data, i))
```

```
    threads.append(t)
```

```
    t.start()
```

```
for t in threads:
```

```
    t.join()
```

```
# Вывод статистики памяти
```

```
monitor_memory()
```

```
print("\n" + "=" * 60)
```

```
print(" 🚩 ПРОГРАММА ЗАВЕРШЕНА")
```

```
print("=" * 60)
```

Этап 3: Условные точки останова

ОТЛАДКА ПРОГРАММЫ (ЭТАП 3: УСЛОВНЫЕ ТОЧКИ ОСТАНОВА)

ИНФОРМАЦИЯ ОБ УСЛОВНЫХ ТОЧКАХ ОСТАНОВА:

1. ● Остановка на элементе 3 (отсутствует 'price')
Условие: `if i == 3`
Где: `process_booking()`, строка ~44
2. ● Остановка при `nights > 4`
Условие: `if item.get('nights', 0) > 4`
Где: `process_booking()`, строка ~51
3. ● Остановка при `price > 1000`
Условие: `if item.get('price', 0) > 1000`
Где: `process_booking()`, строка ~58
4. ● Остановка при заполнении кеша
Условие: `if len(CACHE) > CACHE_LIMIT - 10`
Где: `process_booking()`, строка ~67
5. ● Остановка только для потока 2
Условие: `if thread_id == 2`
Где: `worker()`, строка ~82

Тестовые данные:

```
0: {'price': 1200, 'nights': 3}
1: {'price': 500, 'nights': 5}
2: {'price': 100, 'nights': 2}
3: {'vip': True, 'nights': 1}
4: {'price': 800, 'nights': 4}
5: {'price': 300, 'nights': 0}
6: {'price': 1500, 'nights': 6}
```

1 ☐ ОДНОПОТОЧНЫЙ ВЫЗОВ:

```
● УСЛОВНАЯ ОСТАНОВКА: price > 1000 в элементе 0
  item = {'price': 1200, 'nights': 3}
> y:\24 исип 2025\уп02\группа 2\позднякова м\день 12\3.py(56)process_booking()
-> breakpoint() # Остановка для дорогих бронирований
(Pdb) |
```

mar.pdbrc – Блокнот

Файл Правка Формат Вид Справка

```
# Запусти программу, затем в pdb:
(Pdb) break process_booking if len(CACHE) > 50
Breakpoint 1 at 1.py:37
(Pdb) break 44 if item.get('price', 0) == 500
Breakpoint 2 at 1.py:44
(Pdb) break calculate_discount if nights > 5
Breakpoint 3 at 1.py:15
(Pdb) c
```

Код:

"""

Вариант 1 — Отладка программы (Список словарей)

ЭТАП 3: Условные точки останова

"""

```
import threading
```

```
import time
```

```
import tracemalloc
```

```
import math
```

```
import sys
```

```
# Глобальный кеш --- утечка памяти
```

```
CACHE = {}
```

```
CACHE_LIMIT = 100
```

```
def calculate_discount(price, nights):
```

```
    """
```

```
    Расчет скидки с логической ошибкой:
```

```
    - При nights > 3: скидка должна быть 20%, но формула неверная
```

```
    """
```

```
# Логическая ошибка: скидка 20% ночью, но формула неправильная
```

```
if nights > 3:
```

```
# Должно быть price * 0.2, а здесь price * (0.2 + nights)
```

```
discount = price * (0.2 + nights) # Ошибка!
```

```
else:
```

```
    discount = 0
```

```
return discount
```

```
def process_booking(data):
```

```
    """
```

```
    Обработка бронирований с многопоточным доступом
```

```
    """
```

```
    for i, item in enumerate(data):
```

```
        #
```

```
=====
=
```

```
    # УСЛОВНАЯ ТОЧКА ОСТАНОВА 1: Останавливаемся только на 4-м
    элементе
```

```
        #
```

```
=====
=
```

```
    if i == 3: # индексация с 0, поэтому 3 = 4-й элемент
```

```
        print(f"\n● УСЛОВНАЯ ОСТАНОВКА: элемент {i} = {item}")
```

```
        print(" (Это элемент без ключа 'price!')")
```

```
        breakpoint() # Остановка только на элементе с ошибкой
```

#

=====

=

УСЛОВНАЯ ТОЧКА ОСТАНОВА 2: Останавливаемся, если nights > 4

#

=====

=

if item.get('nights', 0) > 4:

print(f"\n● УСЛОВНАЯ ОСТАНОВКА: nights > 4 в элементе {i}")

print(f" item = {item}")

breakpoint() # Остановка, если nights > 4 (должна быть скидка)

#

=====

=

УСЛОВНАЯ ТОЧКА ОСТАНОВА 3: Останавливаемся, если price >
1000

#

=====

=

if item.get('price', 0) > 1000:

print(f"\n● УСЛОВНАЯ ОСТАНОВКА: price > 1000 в элементе {i}")

print(f" item = {item}")

breakpoint() # Остановка для дорогих бронирований

```
price = item['price'] # KeyError при отсутствии ключа!
```

```
nights = item['nights']
```

```
#
```

```
=====
```

```
=
```

```
# УСЛОВНАЯ ТОЧКА ОСТАНОВА 4: Останавливаемся перед  
кешированием
```

```
#
```

```
=====
```

```
=
```

```
if len(CACHE) > CACHE_LIMIT - 10: # Почти достигнут лимит
```

```
    print(f"\n🚫 УСЛОВНАЯ ОСТАНОВКА: кеш почти полон  
( {len(CACHE)} / {CACHE_LIMIT} )")
```

```
    breakpoint() # Проверка перед переполнением кеша
```

```
# Утечка: кешируем все, даже если не нужно
```

```
CACHE[f'{price}_{nights}'] = calculate_discount(price, nights)
```

```
return CACHE
```

```
def worker(data, thread_id):
```

```
    """Функция для потока"""
```

```
    print(f"🧵 Поток {thread_id} начал работу")
```

```

#
=====

=

# УСЛОВНАЯ ТОЧКА ОСТАНОВА 5: Останавливаемся только для
потока 2

#
=====

=

if thread_id == 2:

    print(f"\n🛑 УСЛОВНАЯ ОСТАНОВКА: поток {thread_id}
(последний)")

    breakpoint() # Отладка последнего потока


result = process_booking(data)

print(f"✅ Поток {thread_id} завершил работу")

return result


def monitor_memory():

    """Мониторинг использования памяти"""

    snapshot = tracemalloc.take_snapshot()

    top_stats = snapshot.statistics('lineno')

    print("\n" + "=" * 60)

```

```

print("📊 СТАТИСТИКА ПАМЯТИ:")

print("=" * 60)

print("\nТоп-5 строк по потреблению памяти:")

for stat in top_stats[:5]:

    print(stat)


return top_stats

```

```

def debug_breakpoints():

    """Вывод информации о точках останова"""

    print("\n" + "=" * 60)

    print("🔗 ИНФОРМАЦИЯ ОБ УСЛОВНЫХ ТОЧКАХ ОСТАНОВА:")

    print("=" * 60)

    print("1. 🟡 Остановка на элементе 3 (отсутствует 'price')")

    print("   Условие: if i == 3")

    print("   Где: process_booking(), строка ~44")

    print()

    print("2. 🟡 Остановка при nights > 4")

    print("   Условие: if item.get('nights', 0) > 4")

    print("   Где: process_booking(), строка ~51")

    print()

    print("3. 🟡 Остановка при price > 1000")

```

```
print(" Условие: if item.get('price', 0) > 1000")

print(" Где: process_booking(), строка ~58")

print()

print("4. ● Остановка при заполнении кеша")

print(" Условие: if len(CACHE) > CACHE_LIMIT - 10")

print(" Где: process_booking(), строка ~67")

print()

print("5. ● Остановка только для потока 2")

print(" Условие: if thread_id == 2")

print(" Где: worker(), строка ~82")

print("=" * 60)


if __name__ == "__main__":

    # Запуск отслеживания памяти

    tracemalloc.start()


    print("=" * 60)

    print("🔍 ОТЛАДКА ПРОГРАММЫ (ЭТАП 3: УСЛОВНЫЕ ТОЧКИ  
ОСТАНОВА)")

    print("=" * 60)


    # Показываем информацию о точках останова

    debug_breakpoints()
```

Тестовые данные с ошибкой

```
test_data = [  
    {'price': 1200, 'nights': 3},    # price > 1000  
    {'price': 500, 'nights': 5},    # nights > 4  
    {'price': 100, 'nights': 2},  
    {'vip': True, 'nights': 1},    # ✗ НЕТ 'price!' (элемент 3)  
    {'price': 800, 'nights': 4},  
    {'price': 300, 'nights': 0},  
    {'price': 1500, 'nights': 6},    # price > 1000 и nights > 4  
]
```

```
print("\n📋 Тестовые данные:")
```

```
for i, item in enumerate(test_data):
```

```
    print(f" {i}: {item}")
```

```
print("\n" + "=" * 60)
```

```
print("🔄 ОДНОПОТОЧНЫЙ ВЫЗОВ:")
```

```
print("=" * 60)
```

```
try:
```

```
    result = process_booking(test_data)
```

```
print(f"\n✅ Результат: {result}")
```

```
print(f"📁 Размер кеша: {len(CACHE)}")
```

```
except Exception as e:
```

```
    print(f"\n❌ ОШИБКА: {e}")
```

```
import traceback
```

```
    traceback.print_exc()
```

```
#
```

```
=====
```

```
=
```

```
# УСЛОВНАЯ ТОЧКА ОСТАНОВА В PDB (интерактивно)
```

```
#
```

```
=====
```

```
=
```

```
print("\n" + "=" * 60)
```

```
print("🔄 МНОГОПОТОЧНЫЙ ВЫЗОВ:")
```

```
print("=" * 60)
```

```
print("\n🔗 Как установить условную точку в pdb:")
```

```
print("  break process_booking if len(CACHE) > 50")
```

```
print("  или")
```

```
print("  break 37 if item.get('price', 0) == 500")
```

```
print("=" * 60)
```

```
threads = []

for i in range(3):

    t = threading.Thread(target=worker, args=(test_data, i))

    threads.append(t)

    t.start()


for t in threads:

    t.join()


# Вывод статистики памяти

monitor_memory()


print("\n" + "=" * 60)

print("🚩 ПРОГРАММА ЗАВЕРШЕНА")

print("=" * 60)
```

Этап 4: Отладка памяти с tracemalloc

```
✓ tracemalloc запущен

Генерация тестовых данных (1000 элементов)...
✓ Сгенерировано 1000 элементов
(Каждый 10-й элемент не имеет ключа 'price')

=====

📷 СНИМОК 1: ДО ВЫПОЛНЕНИЯ

=====

📊 СТАТИСТИКА ПАМЯТИ (tracemalloc):

=====

📁 ТОП-5 ФАЙЛОВ ПО ПОТРЕБЛЕНИЮ ПАМЯТИ:
-----
1. Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/4.py:
  Размер: 239.28 КБ (245024 байт)
  Количество объектов: 3636
  Средний размер: 67.4 байт
2. threading.py:
  Размер: 1.30 КБ (1336 байт)
  Количество объектов: 11
  Средний размер: 121.5 байт
3. grpc.py:
  Размер: 0.57 КБ (587 байт)
  Количество объектов: 7
  Средний размер: 83.9 байт
4. run.py:
  Размер: 0.06 КБ (64 байт)
  Количество объектов: 1
  Средний размер: 64.0 байт

📄 ТОП-5 СТРОК ПО ПОТРЕБЛЕНИЮ ПАМЯТИ:
-----
1. Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/4.py:261
  Размер: 179.50 КБ (183808 байт)
  Количество объектов: 1997
  Средний размер: 92.0 байт
2. Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/4.py:262
  Размер: 27.97 КБ (28640 байт)
  Количество объектов: 895
  Средний размер: 32.0 байт
3. Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/4.py:260
  Размер: 23.22 КБ (23776 байт)
  Количество объектов: 743
  Средний размер: 32.0 байт
4. Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/4.py:269
  Размер: 8.59 КБ (8800 байт)
  Количество объектов: 1
  Средний размер: 8800.0 байт
5. threading.py:297
  Размер: 0.74 КБ (760 байт)
  Количество объектов: 2
  Средний размер: 380.0 байт

📊 ОБЩАЯ СТАТИСТИКА:
Всего памяти: 0.24 МБ
Всего объектов: 3655
```

Код:

```
"""
```

Вариант 1 — Отладка программы (Список словарей)

ЭТАП 4: Отладка памяти с tracemalloc

```
"""
```

```
import threading
```

```
import time
```

```
import tracemalloc
```

```
import math
```

```
import sys
```

```
import gc
```

```
from datetime import datetime
```

```
#
```

```
=====
```

```
=
```

```
# Глобальный кеш --- потенциальная утечка памяти
```

```
#
```

```
=====
```

```
=
```

```
CACHE = {}
```

```
CACHE_LIMIT = 100
```

```
#
```

```
=====
```

```
=
```

```
# СЧЁТЧИКИ ДЛЯ МОНИТОРИНГА
```

```
#
```

```
=====
```

```
=
```

```
call_counter = 0
```

```
cache_hits = 0
```

```
cache_misses = 0
```

```
def calculate_discount(price, nights):
```

```
    """
```

```
    Расчет скидки с логической ошибкой:
```

```
    - При nights > 3: скидка должна быть 20%, но формула неверная
```

```
    """
```

```
    global call_counter
```

```
    call_counter += 1
```

```
    # Логическая ошибка: скидка 20% ночью, но формула неправильная
```

```
    if nights > 3:
```

```
        # Должно быть price * 0.2, а здесь price * (0.2 + nights)
```

```
        discount = price * (0.2 + nights) # Ошибка!
```

```
    else:
```

```
        discount = 0
```

```
    return discount
```

```
def process_booking(data):
```

```
    """
```

```
    Обработка бронирований с многопоточным доступом
```

```
    """
```

```
    global CACHE, cache_hits, cache_misses
```

```
for i, item in enumerate(data):
```

```
    #
```

```
=====
```

```
=
```

```
    # ТОЧКА ОСТАНОВА ДЛЯ ПАМЯТИ: Проверка перед обработкой
```

```
    #
```

```
=====
```

```
=
```

```
    print(f"\n🔍 Элемент {i}: {item}")
```

```
    # Безопасный доступ к ключам (исправление KeyError)
```

```
    price = item.get('price', 0)
```

```
    nights = item.get('nights', 0)
```

```
    key = f'{price}_{nights}'
```

```
    # Проверка кеша
```

```
    if key in CACHE:
```

```
        cache_hits += 1
```

```
        print(f"✅ Кеш-хит: {key} = {CACHE[key]}")
```

```
    #
```

```
=====
```

```
=
```

ТОЧКА ОСТАНОВА ДЛЯ ПАМЯТИ: Кеш-хит

#

=====

=

if len(CACHE) % 10 == 0: # Каждые 10 элементов

print(f" 📊 Размер кеша: {len(CACHE)}, Хиты: {cache_hits},
Пропуски: {cache_misses}")

breakpoint() # Остановка для проверки состояния кеша

continue

cache_misses += 1

#

=====

=

ТОЧКА ОСТАНОВА ДЛЯ ПАМЯТИ: Перед вычислением

#

=====

=

print(f" 📝 Вычисление для: {key}")

breakpoint() # Остановка перед вычислением

discount = calculate_discount(price, nights)

#

=====

=

ТОЧКА ОСТАНОВА ДЛЯ ПАМЯТИ: После вычисления

#

=====

=

print(f' 📄 Сохранение в кеш: {key} = {discount}')

Утечка: кешируем все, даже если не нужно

CACHE[key] = discount

#

=====

=

ТОЧКА ОСТАНОВА ДЛЯ ПАМЯТИ: Проверка размера кеша

#

=====

=

if len(CACHE) % 20 == 0: # Каждые 20 элементов

print(f'\n ⬤ ПРОВЕРКА ПАМЯТИ: Размер кеша = {len(CACHE)}')

print(" 📄 Вызов tracemalloc для снимка памяти")

monitor_memory() # Замер памяти при росте кеша

breakpoint() # Остановка для анализа

```

print(f"\n📊 ИТОГОВАЯ СТАТИСТИКА КЕША:")

print(f"  Размер: {len(CACHE)}")

print(f"  Хиты: {cache_hits}")

print(f"  Пропуски: {cache_misses}")

print(f"  Коэффициент хитов: {(cache_hits / (cache_hits + cache_misses) *
100):.1f}%")

```

```

return CACHE

```

```

def worker(data, thread_id):

    """Функция для потока"""

    print(f"🌀 Поток {thread_id} начал работу")

    result = process_booking(data)

    print(f"✅ Поток {thread_id} завершил работу")

    return result

```

```

#

```

```

=====

```

```

=

```

```

# ФУНКЦИИ МОНИТОРИНГА ПАМЯТИ

```

```

#

```

```

=====

```

```

=

```

```

def monitor_memory():

    """Мониторинг использования памяти с tracemalloc"""

    print("\n" + "=" * 70)

    print("📊 СТАТИСТИКА ПАМЯТИ (tracemalloc):")

    print("=" * 70)

    try:

        # Получаем снимок текущей памяти

        snapshot = tracemalloc.take_snapshot()

        # Статистика по файлам

        print("\n📁 ТОП-5 ФАЙЛОВ ПО ПОТРЕБЛЕНИЮ ПАМЯТИ:")

        print("-" * 70)

        top_stats_files = snapshot.statistics('filename')

        for i, stat in enumerate(top_stats_files[:5], 1):

            size_kb = stat.size / 1024

            count = stat.count

            file_name = stat.traceback[0].filename.split("\\")[-1]

            print(f" {i}. {file_name}:")

            print(f"    Размер: {size_kb:.2f} КБ ({stat.size} байт)")

            print(f"    Количество объектов: {count}")

```

```

print(f"    Средний размер: {stat.size / count if count else 0:.1f} байт")

# Статистика по строкам кода

print("\n📄 ТОП-5 СТРОК ПО ПОТРЕБЛЕНИЮ ПАМЯТИ:")

print("-" * 70)

top_stats_lines = snapshot.statistics('lineno')

for i, stat in enumerate(top_stats_lines[:5], 1):

    size_kb = stat.size / 1024

    count = stat.count

    # Получаем информацию о строке

    tb = stat.traceback

    frame = tb[0]

    file_name = frame.filename.split("\\")[-1]

    line_no = frame.lineno

    print(f"  {i}. {file_name}:{line_no}")

    print(f"    Размер: {size_kb:.2f} КБ ({stat.size} байт)")

    print(f"    Количество объектов: {count}")

    print(f"    Средний размер: {stat.size / count if count else 0:.1f} байт")

# Общая статистика

total_size = sum(stat.size for stat in top_stats_files)

total_size_mb = total_size / (1024 * 1024)

```

```
print("\n 📊 ОБЩАЯ СТАТИСТИКА:")
```

```
print(f"  Всего памяти: {total_size_mb:.2f} МБ")
```

```
print(f"  Всего объектов: {sum(stat.count for stat in top_stats_files)}")
```

```
return snapshot
```

```
except Exception as e:
```

```
print(f" ⚠ Ошибка мониторинга памяти: {e}")
```

```
return None
```

```
def compare_memory_snapshots(snapshot1, snapshot2):
```

```
    """Сравнение двух снимков памяти"""
```

```
    print("\n" + "=" * 70)
```

```
    print(" 📊 СРАВНЕНИЕ СНИМКОВ ПАМЯТИ:")
```

```
    print("=" * 70)
```

```
    try:
```

```
        # Вычисляем разницу между снимками
```

```
        diff = snapshot2.compare_to(snapshot1, 'lineno')
```

```
    print("\n 🕒 ТОП-5 УТЕЧЕК ПАМЯТИ (НОВЫЕ ОБЪЕКТЫ):")
```

```
    print("-" * 70)
```

```

for i, stat in enumerate(diff[:5], 1):

    if stat.size_diff > 0: # Только положительная разница

        size_diff_kb = stat.size_diff / 1024

        count_diff = stat.count_diff


        # Получаем информацию о строке

        tb = stat.traceback

        frame = tb[0]

        file_name = frame.filename.split('\\')[-1]

        line_no = frame.lineno


        print(f" {i}. {file_name}:{line_no}")

        print(f"    Увеличение размера: +{size_diff_kb:.2f} КБ")

        print(f"    Увеличение объектов: +{count_diff}")

        print(f"    Текущий размер: {stat.size / 1024:.2f} КБ")


    return diff


except Exception as e:

    print(f" ⚠ Ошибка сравнения: {e}")

    return None

```

```

def get_cache_memory_info():

    """Информация о памяти, занимаемой кешем"""

    import sys

    if not CACHE:

        return

    print("\n" + "=" * 70)

    print("  ИНФОРМАЦИЯ О КЕШЕ:")

    print("=" * 70)

    try:

        # Размер кеша в байтах

        cache_size_bytes = sys.getsizeof(CACHE)

        # Размер всех ключей и значений

        keys_size = sum(sys.getsizeof(k) for k in CACHE.keys())

        values_size = sum(sys.getsizeof(v) for v in CACHE.values())

        total_size = cache_size_bytes + keys_size + values_size

        print(f"  Количество элементов: {len(CACHE)}")

        print(f"  Размер словаря: {cache_size_bytes / 1024:.2f} КБ")

```

```
print(f"  Размер ключей: {keys_size / 1024:.2f} КБ")

print(f"  Размер значений: {values_size / 1024:.2f} КБ")

print(f"  Общий размер: {total_size / 1024:.2f} КБ ({total_size /
(1024*1024):.4f} МБ)")
```

```
except Exception as e:
```

```
    print(f" ⚠ Ошибка: {e}")
```

```
#
```

```
=====
```

```
=
```

```
# ГЛАВНАЯ ФУНКЦИЯ
```

```
#
```

```
=====
```

```
=
```

```
if __name__ == "__main__":
```

```
    #
```

```
=====
```

```
=
```

```
    # ЗАПУСК ОТСЛЕЖИВАНИЯ ПАМЯТИ
```

```
    #
```

```
=====
```

```
=
```

```
    print("=" * 70)
```

```
print("🔧 ОТЛАДКА ПРОГРАММЫ (ЭТАП 4: ОТЛАДКА ПАМЯТИ С  
tracemalloc)")
```

```
print("=" * 70)
```

```
# Запускаем tracemalloc
```

```
tracemalloc.start()
```

```
print("\n✅ tracemalloc запущен")
```

```
#
```

```
=====
```

```
=
```

```
# СОЗДАНИЕ ТЕСТОВЫХ ДАННЫХ (БОЛЬШОЙ ОБЪЁМ ДЛЯ  
УТЕЧКИ)
```

```
#
```

```
=====
```

```
=
```

```
print("\n📋 Генерация тестовых данных (1000 элементов)...")
```

```
test_data = []
```

```
for i in range(1000):
```

```
    item = {
```

```
        'price': (i + 1) * 50, # 50, 100, 150, ...
```

```
        'nights': (i % 7) + 1, # 1-7
```

```
        'id': i
```

```
}  
  
# Добавляем элементы без 'price' для проверки  
  
if i % 10 == 5: # Каждый 10-й элемент без 'price'  
  
    del item['price']  
  
test_data.append(item)
```

```
print(f"✅ Сгенерировано {len(test_data)} элементов")
```

```
print(" (Каждый 10-й элемент не имеет ключа 'price')")
```

```
#
```

```
=====
```

```
=
```

```
# ПЕРВЫЙ СНИМОК ПАМЯТИ (ДО ВЫПОЛНЕНИЯ)
```

```
#
```

```
=====
```

```
=
```

```
print("\n" + "=" * 70)
```

```
print("📷 СНИМОК 1: ДО ВЫПОЛНЕНИЯ")
```

```
print("=" * 70)
```

```
snapshot_before = monitor_memory()
```

```
get_cache_memory_info()
```

#

=

ТОЧКА ОСТАНОВА: ПЕРЕД ВЫПОЛНЕНИЕМ

#

=

print("\n□ Остановка перед выполнением (проверка памяти)")

breakpoint()

#

=

ВЫПОЛНЕНИЕ ПРОГРАММЫ

#

=

print("\n" + "=" * 70)

print("☑ ВЫПОЛНЕНИЕ ПРОГРАММЫ:")

print("=" * 70)

try:

print("\n□ Однопоточный вызов...")

print("-" * 70)

```
result = process_booking(test_data)
```

```
print(f"\n✅ Результат: {len(result)} элементов в кеше")
```

```
except Exception as e:
```

```
print(f"\n❌ ОШИБКА: {e}")
```

```
import traceback
```

```
traceback.print_exc()
```

```
#
```

```
=====
```

```
=
```

```
# ВТОРОЙ СНИМОК ПАМЯТИ (ПОСЛЕ ВЫПОЛНЕНИЯ)
```

```
#
```

```
=====
```

```
=
```

```
print("\n" + "=" * 70)
```

```
print("📷 СНИМОК 2: ПОСЛЕ ВЫПОЛНЕНИЯ")
```

```
print("=" * 70)
```

```
snapshot_after = monitor_memory()
```

```
get_cache_memory_info()
```

```
#
```

```
=====
```

```
=
```

```
# СРАВНЕНИЕ СНИМКОВ (ВЫЯВЛЕНИЕ УТЕЧЕК)
```

```
#
```

```
=====
```

```
if snapshot_before and snapshot_after:
```

```
    diff = compare_memory_snapshots(snapshot_before, snapshot_after)
```

```
# Вывод топ-строк с утечками
```

```
print("\n" + "=" * 70)
```

```
print("🔍 ВЫЯВЛЕННЫЕ УТЕЧКИ ПАМЯТИ:")
```

```
print("=" * 70)
```

```
# Анализ кеша
```

```
if len(CACHE) > 100:
```

```
    print(f"\n⚠️ ОБНАРУЖЕНА УТЕЧКА ПАМЯТИ!")
```

```
    print(f"    Размер кеша: {len(CACHE)} (превышает лимит  
{CACHE_LIMIT})")
```

```
    print(f"    Память кеша: ~{sys.getsizeof(CACHE) / (1024*1024):.2f} МБ")
```

```
    print("\n💡 Рекомендация:")
```

```
    print("    - Использовать lru_cache(maxsize=128)")
```

```
    print("    - Добавить очистку кеша при превышении лимита")
```

```
    print("    - Использовать threading.Lock() для синхронизации")
```

#

=====

=

ТОЧКА ОСТАНОВА: ПОСЛЕ АНАЛИЗА

#

=====

=

print("\n ☐ Остановка после анализа памяти")

breakpoint()

#

=====

=

ОСТАНОВКА TRACEMALLOC

#

=====

=

tracemalloc.stop()

print("\n ☒ tracemalloc остановлен")

#

=====

=

ИТОГОВАЯ СТАТИСТИКА

#

=

```
print("\n" + "=" * 70)
```

```
print("🚩 ИТОГОВАЯ СТАТИСТИКА:")
```

```
print("=" * 70)
```

```
print(f"📄 Размер кеша: {len(CACHE)}")
```

```
print(f"📊 Обработано элементов: {len(test_data)}")
```

```
print(f"✅ Всего вызовов calculate_discount: {call_counter}")
```

```
print(f"📊 Хиты кеша: {cache_hits}")
```

```
print(f"📊 Пропуски кеша: {cache_misses}")
```

#

=

```
# ОЧИСТКА (ДЛЯ ПРЕДОТВРАЩЕНИЯ УТЕЧЕК ПРИ СЛЕДУЮЩЕМ  
ЗАПУСКЕ)
```

#

=

```
CACHE.clear()
```

```
gc.collect()
```

```
print(f"\n🧹 Кеш очищен. Сборщик мусора вызван.")
```

```
print(f" Текущий размер кеша: {len(CACHE)}")
```

```
print("\n" + "=" * 70)
```

```
print("✅ ПРОГРАММА ЗАВЕРШЕНА")
```

```
print("=" * 70)
```

Этап 5: Исправление логики

```
"""
```

Вариант 1 — Отладка программы (Список словарей)

ЭТАП 5: Исправление логики

```
"""
```

```
import threading
```

```
import time
```

```
import tracemalloc
```

```
import math
```

```
import sys
```

```
import gc
```

```
from datetime import datetime
```

```
from functools import lru_cache
```

```
#
```

```
=====
```

```
=
```

```
# ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ
```

```
#
```

```
=====
```

```
=
```

```
CACHE = {}
```

```
CACHE_LIMIT = 100
```

```
cache_lock = threading.Lock()
```

```
# Счётчики для мониторинга
```

```
call_counter = 0
```

```
cache_hits = 0
```

```
cache_misses = 0
```

```
error_counter = 0
```

```
#
```

```
=====
```

```
=
```

```
# ИСПРАВЛЕННАЯ ФУНКЦИЯ calculate_discount
```

```
#
```

```
=====
```

```
=
```

```
@lru_cache(maxsize=128)
```

```
def calculate_discount(price, nights):
```

```
    """
```

```
    Расчёт скидки (ИСПРАВЛЕННАЯ ВЕРСИЯ)
```

✗ БЫЛО (ОШИБОЧНО):

```
if nights > 3:
```

```
    discount = price * (0.2 + nights) # Ошибка!
```

✓ СТАЛО (ПРАВИЛЬНО):

```
if nights > 3:
```

```
    discount = price * 0.2 # Скидка 20%
```

Аргументы:

price (float): стоимость бронирования

nights (int): количество ночей

Возвращает:

float: размер скидки

"""

global call_counter

call_counter += 1

#

=====

=

ИСПРАВЛЕНИЕ 1: Защита от отрицательных значений

```

#
=====

=

if price < 0:

    print(f' ⚠ Предупреждение: отрицательная цена {price}, устанавливаем
0")

    price = 0


if nights < 0:

    print(f' ⚠ Предупреждение: отрицательное количество ночей {nights},
устанавливаем 0")

    nights = 0


#
=====

=

# ИСПРАВЛЕНИЕ 2: Правильный расчёт скидки

#
=====

=

if nights > 3:

    # Скидка 20% для длительных бронирований

    discount = price * 0.2

else:

```

```
discount = 0.0
```

```
#
```

```
=====
```

```
=
```

```
# ИСПРАВЛЕНИЕ 3: Защита от переполнения
```

```
#
```

```
=====
```

```
=
```

```
if discount > price:
```

```
    print(f' ⚠ Предупреждение: скидка {discount} больше цены {price},  
ограничиваем")
```

```
    discount = price * 0.9 # Максимум 90% скидки
```

```
return round(discount, 2) # Округляем до 2 знаков
```

```
#
```

```
=====
```

```
=
```

```
# ИСПРАВЛЕННАЯ ФУНКЦИЯ process_booking
```

```
#
```

```
=====
```

```
=
```

```
def process_booking(data):
```

```
    """
```

Обработка бронирований с исправлениями

✗ ОШИБКИ БЫЛИ:

1. KeyError при отсутствии ключа 'price'
2. Утечка памяти (неограниченный кеш)
3. Нет синхронизации при многопоточном доступе
4. Нет проверки данных

✓ ИСПРАВЛЕНИЯ:

1. Использование .get() для безопасного доступа
2. Ограничение размера кеша (CACHE_LIMIT)
3. Добавлена блокировка threading.Lock()
4. Добавлена валидация данных

"""

```
global CACHE, cache_hits, cache_misses, error_counter
```

```
print("\n" + "=" * 60)
```

```
print("📝 ОБРАБОТКА ДАННЫХ (ИСПРАВЛЕННАЯ ВЕРСИЯ)")
```

```
print("=" * 60)
```

```
result = []
```

```
processed_count = 0
```

```
skipped_count = 0
```

```
for i, item in enumerate(data):
```

```
    #
```

```
=====
=
```

```
    # ИСПРАВЛЕНИЕ 1: Безопасный доступ к ключам
```

```
    #
```

```
=====
=
```

```
    price = item.get('price', 0)
```

```
    nights = item.get('nights', 0)
```

```
    #
```

```
=====
=
```

```
    # ИСПРАВЛЕНИЕ 4: Валидация данных
```

```
    #
```

```
=====
=
```

```
    if price <= 0:
```

```
        print(f"⚠ Элемент {i}: цена {price} <= 0, пропускаем")
```

```
        skipped_count += 1
```

```
        continue
```

```
if nights <= 0:
```

```
    print(f"⚠ Элемент {i}: ночей {nights} <= 0, устанавливаем 1")
```

```
    nights = 1
```

```
# Проверка на наличие дополнительных полей
```

```
if 'vip' in item:
```

```
    print(f" Элемент {i}: VIP клиент")
```

```
#
```

```
=====
```

```
# ИСПРАВЛЕНИЕ 2: Работа с кешем с ограничением
```

```
#
```

```
=====
```

```
key = f'{price}_{nights}'
```

```
# Проверка кеша с блокировкой
```

```
with cache_lock:
```

```
    if key in CACHE:
```

```
        cache_hits += 1
```

```
        discount = CACHE[key]
```

```
        print(f"✅ Кеш-хит {i}: {key} = {discount}")
```

else:

cache_misses += 1

#

=====

=

ИСПРАВЛЕНИЕ 3: Ограничение размера кеша

#

=====

=

if len(CACHE) >= CACHE_LIMIT:

print(f" ⚠ Кеш достиг лимита {CACHE_LIMIT}, очищаем")

CACHE.clear()

print(f" 🧹 Кеш очищен")

Вычисляем скидку

try:

discount = calculate_discount(price, nights)

CACHE[key] = discount

print(f" 📄 Сохранено в кеш {i}: {key} = {discount}")

except Exception as e:

error_counter += 1

print(f" ❌ Ошибка вычисления: {e}")

```
discount = 0.0
```

```
#
```

```
=====
```

```
=
```

```
# ТОЧКА ОСТАНОВА: Проверка результата
```

```
#
```

```
=====
```

```
=
```

```
if discount > price * 0.5: # Скидка больше 50% - подозрительно
```

```
    print(f" ⚠ Подозрительно большая скидка: {discount} (цена: {price})")
```

```
    breakpoint() # Остановка для проверки
```

```
result.append({
```

```
    'price': price,
```

```
    'nights': nights,
```

```
    'discount': discount,
```

```
    'total': price - discount
```

```
})
```

```
processed_count += 1
```

#

=====

=

ИТОГОВАЯ СТАТИСТИКА

#

=====

=

```
print("\n" + "=" * 60)
```

```
print("  СТАТИСТИКА ОБРАБОТКИ:")
```

```
print("=" * 60)
```

```
print(f"  Всего элементов: {len(data)}")
```

```
print(f"  Обработано: {processed_count}")
```

```
print(f"  Пропущено: {skipped_count}")
```

```
print(f"  Ошибок: {error_counter}")
```

```
print(f"  Размер кеша: {len(CACHE)}")
```

```
print(f"  Хиты кеша: {cache_hits}")
```

```
print(f"  Пропуски кеша: {cache_misses}")
```

```
print(f"  Коэффициент хитов: {(cache_hits / (cache_hits + cache_misses) *  
100 if (cache_hits + cache_misses) > 0 else 0):.1f}%")
```

```
return result
```

#

=====

=

ИСПРАВЛЕННАЯ ФУНКЦИЯ worker (многопоточность)

#

=====

=

def worker(data, thread_id):

"""Функция для потока с обработкой ошибок"""

print(f'🌀 Поток {thread_id} начал работу')

try:

result = process_booking(data)

print(f'✅ Поток {thread_id} завершил работу, обработано {len(result)}
элементов')

return result

except Exception as e:

print(f'❌ Поток {thread_id} ошибка: {e}')

import traceback

traceback.print_exc()

return None

#

=====

=

ФУНКЦИИ МОНИТОРИНГА

#

=====

=

def monitor_memory():

"""Мониторинг использования памяти"""

print("\n" + "=" * 60)

print("  СТАТИСТИКА ПАМЯТИ:")

print("=" * 60)

try:

snapshot = tracemalloc.take_snapshot()

top_stats = snapshot.statistics('lineno')

print("\nТоп-5 строк по потреблению памяти:")

for stat in top_stats[:5]:

size_kb = stat.size / 1024

print(f" {stat}")

print(f" Размер: {size_kb:.2f} КБ")

print(f" Количество: {stat.count}")

```
    return snapshot
```

```
except Exception as e:
```

```
    print(f' ⚠ Ошибка мониторинга: {e}')
```

```
    return None
```

```
def test_scenarios():
```

```
    """Тестирование различных сценариев"""
```

```
    print("\n" + "=" * 60)
```

```
    print(" 🛠 ТЕСТИРОВАНИЕ СЦЕНАРИЕВ:")
```

```
    print("=" * 60)
```

```
    test_cases = [
```

```
        # (price, nights, expected_discount, description)
```

```
        (500, 5, 100.0, "Скидка 20% при nights > 3"),
```

```
        (1000, 3, 0.0, "Нет скидки при nights <= 3"),
```

```
        (300, 0, 0.0, "0 ночей (корректировка до 1)"),
```

```
        (-100, 5, 0.0, "Отрицательная цена"),
```

```
        (1000, -1, 0.0, "Отрицательные ночи"),
```

```
        (10000, 10, 2000.0, "Большая цена и много ночей"),
```

```
    ]
```

```
    for price, nights, expected, description in test_cases:
```

```

result = calculate_discount(price, nights)

status = "✅" if result == expected else "❌"

print(f" {status} {description}:")

print(f"    price={price}, nights={nights} -> {result} (ожидалось
{expected})")

```

```

#

```

```

=====

```

```

=

```

```

# ГЛАВНАЯ ФУНКЦИЯ

```

```

#

```

```

=====

```

```

=

```

```

if __name__ == "__main__":

```

```

    # Запуск отслеживания памяти

```

```

    tracemalloc.start()

```

```

    print("=" * 70)

```

```

    print("🔧 ОТЛАДКА ПРОГРАММЫ (ЭТАП 5: ИСПРАВЛЕНИЕ
ЛОГИКИ)")

```

```

    print("=" * 70)

```

```

# Тестирование сценариев

```

```

test_scenarios()

```

#

=

ТЕСТОВЫЕ ДАННЫЕ (ВСЕ ИСПРАВЛЕНЫ)

#

=

print("\n📋 ТЕСТОВЫЕ ДАННЫЕ (ИСПРАВЛЕННЫЕ):")

print("=" * 60)

test_data = [

 {'price': 1200, 'nights': 3}, # ☒ Нет скидки (3 ночи)

 {'price': 500, 'nights': 5}, # ☒ Скидка 20% = 100

 {'price': 100, 'nights': 2}, # ☒ Нет скидки

 {'price': 0, 'vip': True, 'nights': 1}, # ☒ Теперь есть 'price' (0)

 {'price': 800, 'nights': 4}, # ☒ Скидка 20% = 160

 {'price': 300, 'nights': 0}, # ☒ 0 ночей (исправится на 1)

 {'price': 1500, 'nights': 6}, # ☒ Скидка 20% = 300

 {'price': 2000, 'nights': 10}, # ☒ Скидка 20% = 400

 {'price': 50, 'nights': 5}, # ☒ Скидка 20% = 10

]

```
for i, item in enumerate(test_data):
```

```
    print(f"    {i}: {item}")
```

```
#
```

```
=====
```

```
=
```

```
# СНИМОК ПАМЯТИ ДО ВЫПОЛНЕНИЯ
```

```
#
```

```
=====
```

```
=
```

```
print("\n📷 СНИМОК ПАМЯТИ ДО ВЫПОЛНЕНИЯ:")
```

```
snapshot_before = monitor_memory()
```

```
#
```

```
=====
```

```
=
```

```
# 1. ОДНОПОТОЧНЫЙ ВЫЗОВ
```

```
#
```

```
=====
```

```
=
```

```
print("\n" + "=" * 70)
```

```
print("1 ☐ ОДНОПОТОЧНЫЙ ВЫЗОВ (ИСПРАВЛЕННЫЙ):")
```

```
print("=" * 70)
```

try:

```
result = process_booking(test_data)
```

```
print("\n 🟢 РЕЗУЛЬТАТЫ ОБРАБОТКИ:")
```

```
print("=" * 60)
```

```
for i, item in enumerate(result):
```

```
    print(f"  {i}: {item}")
```

```
print("\n 📊 ИТОГОВАЯ СТАТИСТИКА:")
```

```
print(f"  Всего результатов: {len(result)}")
```

```
total_discount = sum(item['discount'] for item in result)
```

```
total_price = sum(item['price'] for item in result)
```

```
print(f"  Общая цена: {total_price}")
```

```
print(f"  Общая скидка: {total_discount}")
```

```
print(f"  Средняя скидка: {total_discount / len(result) if result else 0:.2f}")
```

except Exception as e:

```
print(f"\n ❌ ОШИБКА: {e}")
```

```
import traceback
```

```
traceback.print_exc()
```

#

=====

=

2. МНОГОПОТОЧНЫЙ ВЫЗОВ

#

=====

=

```
print("\n" + "=" * 70)
```

```
print("2 МНОГОПОТОЧНЫЙ ВЫЗОВ (ИСПРАВЛЕННЫЙ):")
```

```
print("=" * 70)
```

```
# Создаём большие данные для многопоточного теста
```

```
large_data = []
```

```
for i in range(100):
```

```
    large_data.append({
        'price': (i + 1) * 100,
        'nights': (i % 7) + 1,
        'id': i
    })
```

```
print(f" Создано {len(large_data)} элементов для многопоточного теста")
```

```
threads = []
```

```
for i in range(3):
```

```
t = threading.Thread(target=worker, args=(large_data, i))
```

```
threads.append(t)
```

```
t.start()
```

```
for t in threads:
```

```
    t.join()
```

```
#
```

```
=====
```

```
=
```

```
# СНИМОК ПАМЯТИ ПОСЛЕ ВЫПОЛНЕНИЯ
```

```
#
```

```
=====
```

```
=
```

```
print("\n📷 СНИМОК ПАМЯТИ ПОСЛЕ ВЫПОЛНЕНИЯ:")
```

```
snapshot_after = monitor_memory()
```

```
#
```

```
=====
```

```
=
```

```
# СРАВНЕНИЕ СНИМКОВ
```

```
#
```

```
=====
```

```
=
```

```
if snapshot_before and snapshot_after:
```

```
print("\n📊 СРАВНЕНИЕ СНИМКОВ (УТЕЧКИ ПАМЯТИ):")
```

```
print("=" * 60)
```

```
diff = snapshot_after.compare_to(snapshot_before, 'lineno')
```

```
has_leak = False
```

```
for stat in diff[:10]:
```

```
    if stat.size_diff > 0:
```

```
        has_leak = True
```

```
        print(f" 🕒 {stat}")
```

```
        print(f"    Увеличение: {stat.size_diff / 1024:.2f} КБ")
```

```
if not has_leak:
```

```
    print(f" ✅ Утечек памяти не обнаружено!")
```

```
#
```

```
=====
```

```
=
```

```
# ИТОГОВАЯ СТАТИСТИКА
```

```
#
```

```
=====
```

```
=
```

```
print("\n" + "=" * 70)
```

```
print(" 🚩 ИТОГОВАЯ СТАТИСТИКА:")
```

```
print("=" * 70)
```

```
print(f'📁 Размер кеша: {len(CACHE)}")
```

```
print(f'📊 Всего вызовов calculate_discount: {call_counter}")
```

```
print(f'🎯 Хиты кеша: {cache_hits}")
```

```
print(f'🎯 Промахи кеша: {cache_misses}")
```

```
print(f'❌ Ошибок: {error_counter}")
```

```
# Очистка
```

```
CACHE.clear()
```

```
gc.collect()
```

```
print(f'\n🧹 Кеш очищен. Сборщик мусора вызван.")
```

```
tracemalloc.stop()
```

```
print("✅ tracemalloc остановлен")
```

```
print("\n" + "=" * 70)
```

```
print("✅ ПРОГРАММА УСПЕШНО ЗАВЕРШЕНА (ВСЕ ОШИБКИ  
ИСПРАВЛЕНЫ)")
```

```
print("=" * 70)
```

ВЫВОД:

ОТЛАДКА ПРОГРАММЫ (ЭТАП 5: ИСПРАВЛЕНИЕ ЛОГИКИ)

=====

=====

=====

=

ТЕСТИРОВАНИЕ СЦЕНАРИЕВ:

=====

=

☒ Скидка 20% при nights > 3:

price=500, nights=5 -> 100.0 (ожидалось 100.0)

☒ Нет скидки при nights <= 3:

price=1000, nights=3 -> 0.0 (ожидалось 0.0)

☒ 0 ночей (корректировка до 1):

price=300, nights=0 -> 0.0 (ожидалось 0.0)

 Предупреждение: отрицательная цена -100, устанавливаем 0

☒ Отрицательная цена:

price=-100, nights=5 -> 0.0 (ожидалось 0.0)

 Предупреждение: отрицательное количество ночей -1, устанавливаем 0

☒ Отрицательные ночи:

price=1000, nights=-1 -> 0.0 (ожидалось 0.0)

✓ Большая цена и много ночей:

price=10000, nights=10 -> 2000.0 (ожидалось 2000.0)

📋 ТЕСТОВЫЕ ДАННЫЕ (ИСПРАВЛЕННЫЕ):

=====

=

0: {'price': 1200, 'nights': 3}

1: {'price': 500, 'nights': 5}

2: {'price': 100, 'nights': 2}

3: {'price': 0, 'vip': True, 'nights': 1}

4: {'price': 800, 'nights': 4}

5: {'price': 300, 'nights': 0}

6: {'price': 1500, 'nights': 6}

7: {'price': 2000, 'nights': 10}

8: {'price': 50, 'nights': 5}

📷 СНИМОК ПАМЯТИ ДО ВЫПОЛНЕНИЯ:

=====

=

📊 СТАТИСТИКА ПАМЯТИ:

=====

=

Топ-5 строк по потреблению памяти:

C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:388: size=15.7 KiB, count=178, average=91 B

Размер: 15.73 КБ

Количество: 178

C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\threading.py:297: size=760 B, count=2, average=380 B

Размер: 0.74 КБ

Количество: 2

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:259: size=624 B, count=7, average=89 B

Размер: 0.61 КБ

Количество: 7

C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:355: size=208 B, count=5, average=42 B

Размер: 0.20 КБ

Количество: 5

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:293: size=184 B, count=2, average=92 B

Размер: 0.18 КБ

Количество: 2

=====

=====

1 ОДНОПОТОЧНЫЙ ВЫЗОВ (ИСПРАВЛЕННЫЙ):

=====

=====

=====

=

 ОБРАБОТКА ДАННЫХ (ИСПРАВЛЕННАЯ ВЕРСИЯ)

=====

=

 Сохранено в кеш 0: $1200_3 = 0.0$

 Сохранено в кеш 1: $500_5 = 100.0$

 Сохранено в кеш 2: $100_2 = 0.0$

 Элемент 3: цена $0 \leq 0$, пропускаем

 Сохранено в кеш 4: $800_4 = 160.0$

 Элемент 5: ночей $0 \leq 0$, устанавливаем 1

 Сохранено в кеш 5: $300_1 = 0.0$

 Сохранено в кеш 6: $1500_6 = 300.0$

 Сохранено в кеш 7: $2000_10 = 400.0$

 Сохранено в кеш 8: $50_5 = 10.0$

=====

=

СТАТИСТИКА ОБРАБОТКИ:

=====

=

Всего элементов: 9

Обработано: 8

Пропущено: 1

Ошибок: 0

Размер кеша: 8

Хиты кеша: 0

Промахи кеша: 8

Коэффициент хитов: 0.0%

РЕЗУЛЬТАТЫ ОБРАБОТКИ:

=====

=

0: {'price': 1200, 'nights': 3, 'discount': 0.0, 'total': 1200.0}

1: {'price': 500, 'nights': 5, 'discount': 100.0, 'total': 400.0}

2: {'price': 100, 'nights': 2, 'discount': 0.0, 'total': 100.0}

3: {'price': 800, 'nights': 4, 'discount': 160.0, 'total': 640.0}

4: {'price': 300, 'nights': 1, 'discount': 0.0, 'total': 300.0}

5: {'price': 1500, 'nights': 6, 'discount': 300.0, 'total': 1200.0}
6: {'price': 2000, 'nights': 10, 'discount': 400.0, 'total': 1600.0}
7: {'price': 50, 'nights': 5, 'discount': 10.0, 'total': 40.0}

ИТОГОВАЯ СТАТИСТИКА:

Всего результатов: 8

Общая цена: 6450

Общая скидка: 970.0

Средняя скидка: 121.25

МНОГОПОТОЧНЫЙ ВЫЗОВ (ИСПРАВЛЕННЫЙ):

Создано 100 элементов для многопоточного теста

 Поток 0 начал работу  Поток 1 начал работу  Поток 2 начал работу

=====

=

=====

=

 ОБРАБОТКА ДАННЫХ (ИСПРАВЛЕННАЯ ВЕРСИЯ)  ОБРАБОТКА
ДАННЫХ (ИСПРАВЛЕННАЯ ВЕРСИЯ)  ОБРАБОТКА ДАННЫХ
(ИСПРАВЛЕННАЯ ВЕРСИЯ)

=====

=====

=====

=====

 Сохранено в кеш 0: 100_1 = 0.0

☒ Кеш-хит 0: 100_1 = 0.0

☒ Кеш-хит 0: 100_1 = 0.0

 Сохранено в кеш 1: 200_2 = 0.0

☒ Кеш-хит 1: 200_2 = 0.0

☒ Кеш-хит 1: 200_2 = 0.0

 Сохранено в кеш 2: $300_3 = 0.0$

 Кеш-хит 2: $300_3 = 0.0$

 Кеш-хит 2: $300_3 = 0.0$

 Сохранено в кеш 3: $400_4 = 80.0$

 Кеш-хит 3: $400_4 = 80.0$

 Кеш-хит 3: $400_4 = 80.0$

 Кеш-хит 4: $500_5 = 100.0$

 Кеш-хит 4: $500_5 = 100.0$

 Кеш-хит 4: $500_5 = 100.0$

 Сохранено в кеш 5: $600_6 = 120.0$

 Кеш-хит 5: $600_6 = 120.0$

 Кеш-хит 5: $600_6 = 120.0$

 Сохранено в кеш 6: $700_7 = 140.0$

 Кеш-хит 6: $700_7 = 140.0$

 Кеш-хит 6: $700_7 = 140.0$

 Сохранено в кеш 7: $800_1 = 0.0$

 Кеш-хит 7: $800_1 = 0.0$

 Кеш-хит 7: $800_1 = 0.0$

 Сохранено в кеш 8: $900_2 = 0.0$

 Кеш-хит 8: $900_2 = 0.0$

✓ Кеш-хит 8: $900_2 = 0.0$

 Сохранено в кеш 9: $1000_3 = 0.0$

✓ Кеш-хит 9: $1000_3 = 0.0$

✓ Кеш-хит 9: $1000_3 = 0.0$

 Сохранено в кеш 10: $1100_4 = 220.0$

✓ Кеш-хит 10: $1100_4 = 220.0$

✓ Кеш-хит 10: $1100_4 = 220.0$

 Сохранено в кеш 11: $1200_5 = 240.0$

✓ Кеш-хит 11: $1200_5 = 240.0$

✓ Кеш-хит 11: $1200_5 = 240.0$

 Сохранено в кеш 12: $1300_6 = 260.0$

✓ Кеш-хит 12: $1300_6 = 260.0$

✓ Кеш-хит 12: $1300_6 = 260.0$

 Сохранено в кеш 13: $1400_7 = 280.0$

✓ Кеш-хит 13: $1400_7 = 280.0$

✓ Кеш-хит 13: $1400_7 = 280.0$

 Сохранено в кеш 14: $1500_1 = 0.0$

✓ Кеш-хит 14: $1500_1 = 0.0$

✓ Кеш-хит 14: $1500_1 = 0.0$

 Сохранено в кеш 15: $1600_2 = 0.0$

✓ Кеш-хит 15: $1600_2 = 0.0$

✓ Кеш-хит 15: $1600_2 = 0.0$

📄 Сохранено в кеш 16: $1700_3 = 0.0$

✓ Кеш-хит 16: $1700_3 = 0.0$

✓ Кеш-хит 16: $1700_3 = 0.0$

📄 Сохранено в кеш 17: $1800_4 = 360.0$

✓ Кеш-хит 17: $1800_4 = 360.0$

✓ Кеш-хит 17: $1800_4 = 360.0$

📄 Сохранено в кеш 18: $1900_5 = 380.0$

✓ Кеш-хит 18: $1900_5 = 380.0$

✓ Кеш-хит 18: $1900_5 = 380.0$

📄 Сохранено в кеш 19: $2000_6 = 400.0$

✓ Кеш-хит 19: $2000_6 = 400.0$

✓ Кеш-хит 19: $2000_6 = 400.0$

📄 Сохранено в кеш 20: $2100_7 = 420.0$

✓ Кеш-хит 20: $2100_7 = 420.0$

✓ Кеш-хит 20: $2100_7 = 420.0$

📄 Сохранено в кеш 21: $2200_1 = 0.0$

✓ Кеш-хит 21: $2200_1 = 0.0$

✓ Кеш-хит 21: $2200_1 = 0.0$

 Сохранено в кеш 22: $2300_2 = 0.0$

☒ Кеш-хит 22: $2300_2 = 0.0$

☒ Кеш-хит 22: $2300_2 = 0.0$

 Сохранено в кеш 23: $2400_3 = 0.0$

☒ Кеш-хит 23: $2400_3 = 0.0$

☒ Кеш-хит 23: $2400_3 = 0.0$

 Сохранено в кеш 24: $2500_4 = 500.0$

☒ Кеш-хит 24: $2500_4 = 500.0$

☒ Кеш-хит 24: $2500_4 = 500.0$

 Сохранено в кеш 25: $2600_5 = 520.0$

☒ Кеш-хит 25: $2600_5 = 520.0$

☒ Кеш-хит 25: $2600_5 = 520.0$

 Сохранено в кеш 26: $2700_6 = 540.0$

☒ Кеш-хит 26: $2700_6 = 540.0$

☒ Кеш-хит 26: $2700_6 = 540.0$

 Сохранено в кеш 27: $2800_7 = 560.0$

☒ Кеш-хит 27: $2800_7 = 560.0$

☒ Кеш-хит 27: $2800_7 = 560.0$

 Сохранено в кеш 28: $2900_1 = 0.0$

☒ Кеш-хит 28: $2900_1 = 0.0$

✓ Кеш-хит 28: $2900_1 = 0.0$

 Сохранено в кеш 29: $3000_2 = 0.0$

✓ Кеш-хит 29: $3000_2 = 0.0$

✓ Кеш-хит 29: $3000_2 = 0.0$

 Сохранено в кеш 30: $3100_3 = 0.0$

✓ Кеш-хит 30: $3100_3 = 0.0$

✓ Кеш-хит 30: $3100_3 = 0.0$

 Сохранено в кеш 31: $3200_4 = 640.0$

✓ Кеш-хит 31: $3200_4 = 640.0$

✓ Кеш-хит 31: $3200_4 = 640.0$

 Сохранено в кеш 32: $3300_5 = 660.0$

✓ Кеш-хит 32: $3300_5 = 660.0$

✓ Кеш-хит 32: $3300_5 = 660.0$

 Сохранено в кеш 33: $3400_6 = 680.0$

✓ Кеш-хит 33: $3400_6 = 680.0$

✓ Кеш-хит 33: $3400_6 = 680.0$

 Сохранено в кеш 34: $3500_7 = 700.0$

✓ Кеш-хит 34: $3500_7 = 700.0$

✓ Кеш-хит 34: $3500_7 = 700.0$

 Сохранено в кеш 35: $3600_1 = 0.0$

✓ Кеш-хит 35: $3600_1 = 0.0$

✓ Кеш-хит 35: $3600_1 = 0.0$

📄 Сохранено в кеш 36: $3700_2 = 0.0$

✓ Кеш-хит 36: $3700_2 = 0.0$

✓ Кеш-хит 36: $3700_2 = 0.0$

📄 Сохранено в кеш 37: $3800_3 = 0.0$

✓ Кеш-хит 37: $3800_3 = 0.0$

✓ Кеш-хит 37: $3800_3 = 0.0$

📄 Сохранено в кеш 38: $3900_4 = 780.0$

✓ Кеш-хит 38: $3900_4 = 780.0$

✓ Кеш-хит 38: $3900_4 = 780.0$

📄 Сохранено в кеш 39: $4000_5 = 800.0$

✓ Кеш-хит 39: $4000_5 = 800.0$

✓ Кеш-хит 39: $4000_5 = 800.0$

📄 Сохранено в кеш 40: $4100_6 = 820.0$

✓ Кеш-хит 40: $4100_6 = 820.0$

✓ Кеш-хит 40: $4100_6 = 820.0$

📄 Сохранено в кеш 41: $4200_7 = 840.0$

✓ Кеш-хит 41: $4200_7 = 840.0$

✓ Кеш-хит 41: $4200_7 = 840.0$

 Сохранено в кеш 42: $4300_1 = 0.0$

☒ Кеш-хит 42: $4300_1 = 0.0$

☒ Кеш-хит 42: $4300_1 = 0.0$

 Сохранено в кеш 43: $4400_2 = 0.0$

☒ Кеш-хит 43: $4400_2 = 0.0$

☒ Кеш-хит 43: $4400_2 = 0.0$

 Сохранено в кеш 44: $4500_3 = 0.0$

☒ Кеш-хит 44: $4500_3 = 0.0$

☒ Кеш-хит 44: $4500_3 = 0.0$

 Сохранено в кеш 45: $4600_4 = 920.0$

☒ Кеш-хит 45: $4600_4 = 920.0$

☒ Кеш-хит 45: $4600_4 = 920.0$

 Сохранено в кеш 46: $4700_5 = 940.0$

☒ Кеш-хит 46: $4700_5 = 940.0$

☒ Кеш-хит 46: $4700_5 = 940.0$

 Сохранено в кеш 47: $4800_6 = 960.0$

☒ Кеш-хит 47: $4800_6 = 960.0$

☒ Кеш-хит 47: $4800_6 = 960.0$

 Сохранено в кеш 48: $4900_7 = 980.0$

☒ Кеш-хит 48: $4900_7 = 980.0$

✓ Кеш-хит 48: $4900_7 = 980.0$

 Сохранено в кеш 49: $5000_1 = 0.0$

✓ Кеш-хит 49: $5000_1 = 0.0$

✓ Кеш-хит 49: $5000_1 = 0.0$

 Сохранено в кеш 50: $5100_2 = 0.0$

✓ Кеш-хит 50: $5100_2 = 0.0$

✓ Кеш-хит 50: $5100_2 = 0.0$

 Сохранено в кеш 51: $5200_3 = 0.0$

✓ Кеш-хит 51: $5200_3 = 0.0$

✓ Кеш-хит 51: $5200_3 = 0.0$

 Сохранено в кеш 52: $5300_4 = 1060.0$

✓ Кеш-хит 52: $5300_4 = 1060.0$

✓ Кеш-хит 52: $5300_4 = 1060.0$

 Сохранено в кеш 53: $5400_5 = 1080.0$

✓ Кеш-хит 53: $5400_5 = 1080.0$

✓ Кеш-хит 53: $5400_5 = 1080.0$

 Сохранено в кеш 54: $5500_6 = 1100.0$

✓ Кеш-хит 54: $5500_6 = 1100.0$

✓ Кеш-хит 54: $5500_6 = 1100.0$

 Сохранено в кеш 55: $5600_7 = 1120.0$

✓ Кеш-хит 55: $5600_7 = 1120.0$

✓ Кеш-хит 55: $5600_7 = 1120.0$

📄 Сохранено в кеш 56: $5700_1 = 0.0$

✓ Кеш-хит 56: $5700_1 = 0.0$

✓ Кеш-хит 56: $5700_1 = 0.0$

📄 Сохранено в кеш 57: $5800_2 = 0.0$

✓ Кеш-хит 57: $5800_2 = 0.0$

✓ Кеш-хит 57: $5800_2 = 0.0$

📄 Сохранено в кеш 58: $5900_3 = 0.0$

✓ Кеш-хит 58: $5900_3 = 0.0$

✓ Кеш-хит 58: $5900_3 = 0.0$

📄 Сохранено в кеш 59: $6000_4 = 1200.0$

✓ Кеш-хит 59: $6000_4 = 1200.0$

✓ Кеш-хит 59: $6000_4 = 1200.0$

📄 Сохранено в кеш 60: $6100_5 = 1220.0$

✓ Кеш-хит 60: $6100_5 = 1220.0$

✓ Кеш-хит 60: $6100_5 = 1220.0$

📄 Сохранено в кеш 61: $6200_6 = 1240.0$

✓ Кеш-хит 61: $6200_6 = 1240.0$

✓ Кеш-хит 61: $6200_6 = 1240.0$

 Сохранено в кеш 62: $6300_7 = 1260.0$

☒ Кеш-хит 62: $6300_7 = 1260.0$

☒ Кеш-хит 62: $6300_7 = 1260.0$

 Сохранено в кеш 63: $6400_1 = 0.0$

☒ Кеш-хит 63: $6400_1 = 0.0$

☒ Кеш-хит 63: $6400_1 = 0.0$

 Сохранено в кеш 64: $6500_2 = 0.0$

☒ Кеш-хит 64: $6500_2 = 0.0$

☒ Кеш-хит 64: $6500_2 = 0.0$

 Сохранено в кеш 65: $6600_3 = 0.0$

☒ Кеш-хит 65: $6600_3 = 0.0$

☒ Кеш-хит 65: $6600_3 = 0.0$

 Сохранено в кеш 66: $6700_4 = 1340.0$

☒ Кеш-хит 66: $6700_4 = 1340.0$

☒ Кеш-хит 66: $6700_4 = 1340.0$

 Сохранено в кеш 67: $6800_5 = 1360.0$

☒ Кеш-хит 67: $6800_5 = 1360.0$

☒ Кеш-хит 67: $6800_5 = 1360.0$

 Сохранено в кеш 68: $6900_6 = 1380.0$

☒ Кеш-хит 68: $6900_6 = 1380.0$

✓ Кеш-хит 68: $6900_6 = 1380.0$

 Сохранено в кеш 69: $7000_7 = 1400.0$

✓ Кеш-хит 69: $7000_7 = 1400.0$

✓ Кеш-хит 69: $7000_7 = 1400.0$

 Сохранено в кеш 70: $7100_1 = 0.0$

✓ Кеш-хит 70: $7100_1 = 0.0$

✓ Кеш-хит 70: $7100_1 = 0.0$

 Сохранено в кеш 71: $7200_2 = 0.0$

✓ Кеш-хит 71: $7200_2 = 0.0$

✓ Кеш-хит 71: $7200_2 = 0.0$

 Сохранено в кеш 72: $7300_3 = 0.0$

✓ Кеш-хит 72: $7300_3 = 0.0$

✓ Кеш-хит 72: $7300_3 = 0.0$

 Сохранено в кеш 73: $7400_4 = 1480.0$

✓ Кеш-хит 73: $7400_4 = 1480.0$

✓ Кеш-хит 73: $7400_4 = 1480.0$

 Сохранено в кеш 74: $7500_5 = 1500.0$

✓ Кеш-хит 74: $7500_5 = 1500.0$

✓ Кеш-хит 74: $7500_5 = 1500.0$

 Сохранено в кеш 75: $7600_6 = 1520.0$

✓ Кеш-хит 75: $7600_6 = 1520.0$

✓ Кеш-хит 75: $7600_6 = 1520.0$

📄 Сохранено в кеш 76: $7700_7 = 1540.0$

✓ Кеш-хит 76: $7700_7 = 1540.0$

✓ Кеш-хит 76: $7700_7 = 1540.0$

📄 Сохранено в кеш 77: $7800_1 = 0.0$

✓ Кеш-хит 77: $7800_1 = 0.0$

✓ Кеш-хит 77: $7800_1 = 0.0$

📄 Сохранено в кеш 78: $7900_2 = 0.0$

✓ Кеш-хит 78: $7900_2 = 0.0$

✓ Кеш-хит 78: $7900_2 = 0.0$

📄 Сохранено в кеш 79: $8000_3 = 0.0$

✓ Кеш-хит 79: $8000_3 = 0.0$

✓ Кеш-хит 79: $8000_3 = 0.0$

📄 Сохранено в кеш 80: $8100_4 = 1620.0$

✓ Кеш-хит 80: $8100_4 = 1620.0$

✓ Кеш-хит 80: $8100_4 = 1620.0$

📄 Сохранено в кеш 81: $8200_5 = 1640.0$

✓ Кеш-хит 81: $8200_5 = 1640.0$

✓ Кеш-хит 81: $8200_5 = 1640.0$

 Сохранено в кеш 82: $8300_6 = 1660.0$

 Кеш-хит 82: $8300_6 = 1660.0$

 Кеш-хит 82: $8300_6 = 1660.0$

 Сохранено в кеш 83: $8400_7 = 1680.0$

 Кеш-хит 83: $8400_7 = 1680.0$

 Кеш-хит 83: $8400_7 = 1680.0$

 Сохранено в кеш 84: $8500_1 = 0.0$

 Кеш-хит 84: $8500_1 = 0.0$

 Кеш-хит 84: $8500_1 = 0.0$

 Сохранено в кеш 85: $8600_2 = 0.0$

 Кеш-хит 85: $8600_2 = 0.0$

 Кеш-хит 85: $8600_2 = 0.0$

 Сохранено в кеш 86: $8700_3 = 0.0$

 Кеш-хит 86: $8700_3 = 0.0$

 Кеш-хит 86: $8700_3 = 0.0$

 Сохранено в кеш 87: $8800_4 = 1760.0$

 Кеш-хит 87: $8800_4 = 1760.0$

 Кеш-хит 87: $8800_4 = 1760.0$

 Сохранено в кеш 88: $8900_5 = 1780.0$

 Кеш-хит 88: $8900_5 = 1780.0$

✓ Кеш-хит 88: $8900_5 = 1780.0$

📄 Сохранено в кеш 89: $9000_6 = 1800.0$

✓ Кеш-хит 89: $9000_6 = 1800.0$

✓ Кеш-хит 89: $9000_6 = 1800.0$

📄 Сохранено в кеш 90: $9100_7 = 1820.0$

✓ Кеш-хит 90: $9100_7 = 1820.0$

✓ Кеш-хит 90: $9100_7 = 1820.0$

📄 Сохранено в кеш 91: $9200_1 = 0.0$

✓ Кеш-хит 91: $9200_1 = 0.0$

✓ Кеш-хит 91: $9200_1 = 0.0$

📄 Сохранено в кеш 92: $9300_2 = 0.0$

✓ Кеш-хит 92: $9300_2 = 0.0$

✓ Кеш-хит 92: $9300_2 = 0.0$

⚠ Кеш достиг лимита 100, очищаем

🧹 Кеш очищен

📄 Сохранено в кеш 93: $9400_3 = 0.0$

✓ Кеш-хит 93: $9400_3 = 0.0$

✓ Кеш-хит 93: $9400_3 = 0.0$

📄 Сохранено в кеш 94: $9500_4 = 1900.0$

✓ Кеш-хит 94: $9500_4 = 1900.0$

✓ Кеш-хит 94: $9500_4 = 1900.0$

📄 Сохранено в кеш 95: $9600_5 = 1920.0$

✓ Кеш-хит 95: $9600_5 = 1920.0$

✓ Кеш-хит 95: $9600_5 = 1920.0$

📄 Сохранено в кеш 96: $9700_6 = 1940.0$

✓ Кеш-хит 96: $9700_6 = 1940.0$

✓ Кеш-хит 96: $9700_6 = 1940.0$

📄 Сохранено в кеш 97: $9800_7 = 1960.0$

✓ Кеш-хит 97: $9800_7 = 1960.0$

✓ Кеш-хит 97: $9800_7 = 1960.0$

📄 Сохранено в кеш 98: $9900_1 = 0.0$

✓ Кеш-хит 98: $9900_1 = 0.0$

✓ Кеш-хит 98: $9900_1 = 0.0$

📄 Сохранено в кеш 99: $10000_2 = 0.0$

=====

= ✓ Кеш-хит 99: $10000_2 = 0.0$

📊 СТАТИСТИКА ОБРАБОТКИ:

=====

= ✓ Кеш-хит 99: $10000_2 = 0.0$

=====

=  СТАТИСТИКА ОБРАБОТКИ:

=====

=

Всего элементов:

100=====

====  СТАТИСТИКА ОБРАБОТКИ:

Обработано: 100 Всего элементов:

100=====

=====

Пропущено: 0 Обработано: 100 Всего элементов: 100

Ошибок: 0 Пропущено: 0 Обработано: 100

Размер кеша: 7 Ошибок: 0 Пропущено: 0

Хиты кеша: 201 Размер кеша: 7 Ошибок: 0

Промахи кеша: 107 Хиты кеша: 201 Размер кеша: 7

Коэффициент хитов: 65.3% Промахи кеша: 107 Хиты кеша: 201

✓ Поток 0 завершил работу, обработано 100 элементов Коэффициент хитов: 65.3% Промахи кеша: 107

✓ Поток 1 завершил работу, обработано 100 элементов Коэффициент хитов: 65.3%

✓ Поток 2 завершил работу, обработано 100 элементов

📷 СНИМОК ПАМЯТИ ПОСЛЕ ВЫПОЛНЕНИЯ:

=

 СТАТИСТИКА ПАМЯТИ:

=

Топ-5 строк по потреблению памяти:

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:342: size=18.8 KiB, count=201, average=96 B

Размер: 18.81 КБ

Количество: 201

C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:388: size=16.1 KiB, count=186, average=89 B

Размер: 16.15 КБ

Количество: 186

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:159: size=15.9 KiB, count=209, average=78 B

Размер: 15.89 КБ

Количество: 209

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:174: size=15.3 KiB, count=169, average=92 B

Размер: 15.27 КБ

Количество: 169

Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:343: size=3136
B, count=98, average=32 B

Размер: 3.06 КБ

Количество: 98

СРАВНЕНИЕ СНИМКОВ (УТЕЧКИ ПАМЯТИ):

=====

=

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:342:
size=18.8 KiB (+18.8 KiB), count=201 (+201), average=96 B

Увеличение: 18.81 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:159:
size=15.9 KiB (+15.9 KiB), count=209 (+209), average=78 B

Увеличение: 15.89 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:174:
size=15.3 KiB (+15.3 KiB), count=169 (+169), average=92 B

Увеличение: 15.27 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:343:
size=3136 B (+3136 B), count=98 (+98), average=32 B

Увеличение: 3.06 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:81:
size=2544 B (+2520 B), count=106 (+105), average=24 B

Увеличение: 2.46 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:178:

size=2496 B (+2496 B), count=104 (+104), average=24 B

Увеличение: 2.44 КБ



C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\threading.p

y:297: size=3040 B (+2280 B), count=8 (+6), average=380 B

Увеличение: 2.23 КБ



C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\tracemalloc

.py:67: size=1792 B (+1792 B), count=28 (+28), average=64 B

Увеличение: 1.75 КБ



C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\tracemalloc

.py:193: size=1200 B (+1200 B), count=25 (+25), average=48 B

Увеличение: 1.17 КБ

● Y:/24 ИСиП 2025/УП02/Группа 2/Позднякова М/день 12/5.py:352:

size=936 B (+936 B), count=3 (+3), average=312 B

Увеличение: 0.91 КБ

=====

=====

🚩 ИТОГОВАЯ СТАТИСТИКА:

=====

=====

 Размер кеша: 7

 Всего вызовов calculate_discount: 111

 Хиты кеша: 201

 Промахи кеша: 107

 Ошибок: 0

 Кеш очищен. Сборщик мусора вызван.

 tracemalloc остановлен

=====

=====

 ПРОГРАММА УСПЕШНО ЗАВЕРШЕНА (ВСЕ ОШИБКИ
ИСПРАВЛЕНЫ)

=====

Root Cause Analysis (Корневые причины ошибок)

№	Ошибка	Причина	Исправление
1	KeyError: 'price'	Прямой доступ к ключу словаря без проверки	Использовать item.get('price', 0)
2	Неверный расчет скидки	Формула price * (0.2 + nights) вместо price * 0.2	Изменить формулу на price * 0.2
3	Утечка памяти	Глобальный кеш без ограничения	Добавить lru_cache(maxsize=128) и очистку при превышении
4	Race condition	Многопоточный доступ без синхронизации	Добавить threading.Lock()

Контрольные вопросы:

A1. Что такое стек вызовов (call stack) в Python и как его увидеть?

Стек вызовов — это последовательность вложенных функций от точки входа до места ошибки, которую можно увидеть в traceback при возникновении исключения или в pdb с помощью команды where.

A2. В чем разница между except: и except Exception as e:?

Пустой except перехватывает абсолютно все исключения, включая системные, что считается антипаттерном, а except Exception as e перехватывает только обычные ошибки и даёт доступ к объекту ошибки для вывода информации.

A3. Как запустить Python-скрипт под отладчиком pdb без изменения исходного кода?

Для запуска скрипта под отладчиком без изменения кода используется команда python -m pdb script.py в командной строке.

A4. Какие основные команды pdb вы знаете и для чего они нужны?

Основные команды pdb: n для выполнения следующей строки, s для входа в функцию, c для продолжения до следующей точки останова, p и pp для печати переменных, l для показа кода, w для просмотра стека и q для выхода из отладчика.

A5. Как установить точку останова (breakpoint) в pdb на определенной строке или функции?

Точку останова можно установить в коде через breakpoint() или import pdb; pdb.set_trace(), а в pdb интерактивно через команду break с указанием номера строки или имени функции, например break 42 или break process_booking.

А6. Что такое утечка памяти в Python и как она может возникнуть?

Утечка памяти — это ситуация, когда программа не освобождает память после использования, и её потребление постоянно растёт из-за глобальных кешей без ограничения, замыканий, не закрытых файлов или ссылок на объекты в циклах.

А7. Какой модуль в Python позволяет отслеживать выделение памяти?

Для отслеживания выделения памяти используется встроенный модуль `tracemalloc`, который позволяет делать снимки памяти и сравнивать их для выявления утечек.

А8. Как вывести красивый traceback ошибки, чтобы понять, где именно произошло исключение?

Красивый traceback выводится с помощью `traceback.print_exc()` в блоке `except`, который показывает полную информацию об ошибке с номерами строк и путём выполнения.

А9. Что такое `RecursionError` и как его избежать?

`RecursionError` возникает при превышении лимита рекурсии (по умолчанию 1000) и избегается путём увеличения лимита через `sys.setrecursionlimit()` или заменой рекурсии на итеративный алгоритм.

А10. Для чего нужна команда `up` в `pdb`?

Команда `up` в `pdb` нужна для перемещения на уровень выше по стеку вызовов, что позволяет исследовать переменные в родительской функции, вызвавшей текущую.

В1. В чем разница между `breakpoint()` и `import pdb; pdb.set_trace()`?

`Breakpoint()` — это современный способ для Python 3.7 и выше, который короче и может быть отключён переменной окружения, а `pdb.set_trace()` работает во всех версиях, но всегда вызывает именно `pdb` и не может быть отключён.

В2. Как в `pdb` продолжить выполнение до того момента, когда некоторая переменная станет равной определенному значению?

Для продолжения выполнения до изменения переменной нужно установить условную точку останова через `break` с условием, например `break 42 if x == 100`, после чего использовать команду `c` для продолжения до выполнения условия.

В3. Почему использование `print()` для отладки считается непрофессиональным и к каким проблемам это может привести?

Использование `print()` непрофессионально, потому что это создаёт мусор в консоли, не гибко в настройке, не имеет уровней важности, остаётся в продакшене и сложно анализировать, в отличие от логирования, которое позволяет фильтровать сообщения по уровням и сохранять их в файлы.

В4. Как использовать `tracemalloc` для поиска утечек памяти в долгоживущем приложении?

Для поиска утечек памяти `tracemalloc` запускается в начале работы, затем периодически делаются снимки памяти через `take_snapshot`, которые сравниваются между собой методом `compare_to` для выявления строк кода, создающих больше всего новых объектов, и при обнаружении утечек добавляется очистка кеша или ограничение его размера.

Вывод о проделанной работе

В ходе выполнения практического задания я освоила основные методы отладки программ на Python, включая использование встроенного отладчика `pdb`, условных точек останова и модуля `tracemalloc` для отслеживания утечек памяти. Мною были успешно выявлены и исправлены четыре типа ошибок: `KeyError` при доступе к словарю, логическая ошибка в расчёте скидки, утечка памяти из-за неограниченного кеша и проблемы многопоточного доступа без синхронизации. Я научилась читать стек вызовов, анализировать `traceback`, устанавливать условные брейкпоинты и создавать конфигурационный файл `.pdbrc` для автоматической загрузки точек останова. В результате работы программа стала полностью работоспособной, защищённой от утечек памяти и корректно обрабатывающей все типы данных.